GLM-5

from Vibe Coding to Agentic Engineering

1

Learning Objectives

- Understand the motivation behind **GLM-5** and its place in modern AI
- Learn how GLM-5 adopts **DeepSeek Sparse Attention (DSA)** for efficient long-context processing
- Explore how GLM-5 adopts **Multi-head Latent Attention (MLA)** for KV cache compression
- Analyze the **Mixture of Experts (MoE)** architecture with sigmoid gating
- Discover GLM-5's unique contributions: **hardware sovereignty**, **Slime RL**, and **agentic engineering**

2

Part 1: The GLM-5 Architecture

3

Introduction to GLM-5

- **GLM-5** is Zhipu AI's flagship open-source LLM (Feb 2026), with **744B total parameters** and **40B active** per token.
- It combines proven techniques from DeepSeek with its own innovations for a state-of-the-art system:
- 1. **DSA** (from DeepSeek): Selectively attends to relevant tokens for efficient long-context processing.
- 2. **MLA** (from DeepSeek): Compresses the Key-Value (KV) cache to save memory during inference.
- 3. **MoE**: 256 routed experts with only 8 active per token, plus 1 shared expert.
- 4. **Original contributions**: Trained on Huawei Ascend chips, Slime async RL, Multi-Token Prediction, agentic engineering focus.

4

The Scaling Challenge

- Traditional dense Transformers scale poorly:
- **Compute Bottleneck**: Every parameter is activated for every token ($O(N)$ compute).
- **Memory Bottleneck**: Attention mechanisms store Keys and Values for all past tokens ($O(L)$ memory per token).
- **Attention Bottleneck**: Standard attention attends to all previous tokens, scaling quadratically with sequence length ($O(L^2)$).
- GLM-5 addresses all three by adopting best-in-class techniques and scaling to 744B parameters.

5

GLM-5's Unique Contributions

- **Hardware Sovereignty**: Trained entirely on **Huawei Ascend** chips using the **MindSpore** framework — no NVIDIA GPUs.
- **Slime Async RL**: Novel asynchronous reinforcement learning infrastructure for efficient post-training alignment.
- **Multi-Token Prediction (MTP)**: Speculative decoding module that predicts multiple future tokens at once.
- **Agentic Engineering**: Designed for long-horizon autonomous tasks with **Agent Mode** and **Thinking Mode**.
- **Scale**: 80 transformer layers, 28.5 trillion training tokens, 200K context window, open-sourced under MIT license.
- **Benchmarks**: 77.8% on SWE-Bench Verified, rivaling Claude Opus 4.5 and GPT-5.2 among open-source models.

6

Part 2: DeepSeek Sparse Attention (DSA)

7

The Attention Problem

- Standard dense attention requires attending to **all** preceding tokens.
- For long contexts (e.g., 200K tokens), this becomes prohibitively expensive.
- Not all past tokens are equally important for predicting the next word.
- **DeepSeek's Solution (adopted by GLM-5)**: Use a lightweight "indexer" to pick only the top k most relevant tokens.

8

DSA: Origin and Integration

- DSA was developed by **DeepSeek**, first introduced in **DeepSeek-V3.1** (based on the **Native Sparse Attention** paper, Best Paper at ACL 2025).
- It replaces full quadratic attention with a **selective sparse** mechanism.
- A separate, small indexer network scores each query against all past keys using a reduced dimension space ($d_{index} = 128$).
- The indexer operates in a **no-grad context**: it's a selection mechanism, not trained via attention gradients.
- Selects the **top-2048** most relevant tokens to attend to, ignoring the rest.

🚀 **Interactive Demo:** dsa_demo.html

9

DSA Scoring Formula

```
function DSAScore(Q, K, W_weights)
  // Q: indexer queries, K: cached indexer keys

  // 1. Calculate base dot product
  base_scores = Q * K^T / sqrt(d_index)

  // 2. Weight by head importance
  head_weights = x * W_weights / sqrt(H)

  // 3. Final score is weighted sum across heads
  final_score = sum(base_scores * head_weights)

  return top_k(final_score, k=2048)
```

10

Part 3: Multi-head Latent Attention (MLA)

11

MLA: Origin and Purpose

- MLA was developed by **DeepSeek**, first introduced in the **DeepSeek-V2** paper.
- During autoregressive generation, we must store the Keys and Values for all past tokens.
- This consumes massive GPU VRAM, severely limiting batch size and context length.
- **MLA Insight**: Compress K and V into a smaller latent representation instead of storing full tensors.

12

MLA Compression Strategy

- **Query Path**: Compress h to c_q (rank 2048), then up-project to full multi-head Queries.
- **KV Path**: Compress h to c_{kv} (rank 512 + 64 RoPE).
- **The Trick**: Only cache the compressed c_{kv} vector!
- During generation, dynamically reconstruct the full Keys and Values from the much smaller cache.
- GLM-5 adopts MLA to achieve immense memory savings while maintaining model performance.

13

MLA Key-Value Projections

PYTHON

```python
# Simplified MLA KV projection path
class MLA_KV(nn.Module):
    def __init__(self, hidden, lora_rank, full_dim):
        # Down-project to low rank (e.g., 512)
        self.down = nn.Linear(hidden, lora_rank)
        self.norm = RMSNorm(lora_rank)
        # Up-project to full heads (e.g., 64 heads * 256 dim)
        self.up = nn.Linear(lora_rank, full_dim)

    def forward(self, x):
        # Compress and cache this tiny tensor!
        c_kv = self.norm(self.down(x))

        # Dynamically reconstruct full K and V when needed
        full_kv = self.up(c_kv)
        return c_kv, full_kv
```

Part 4: Mixture of Experts (MoE)

15

Dense vs. Sparse Processing

- **Dense MLP**: Every token passes through the same massive Feed-Forward Network.
- **Mixture of Experts (MoE)**: Replace the large FFN with many smaller "Expert" networks.
- A "Router" analyzes each token and sends it to only a few relevant experts.
- In GLM-5: **256 routed experts**, but only **8 are active** per token, plus **1 shared expert**.
- Dramatically increases parameter count (model capacity) without increasing inference FLOPs.

16

GLM-5 Routing Mechanism

- Unlike older models using Softmax routing, GLM-5 uses **Sigmoid Gating**.
- Each expert is scored independently: $g_i = \sigma(W_{gate}x)$.
- Allows tokens to activate multiple experts independently, rather than forcing them to compete.
- A bias correction term helps balance the load so all experts receive roughly equal traffic.
- Includes **1 Shared Expert** that processes *every* token, capturing general knowledge.

🚀 **Interactive Demo:** moe_demo.html

17

GLM-5 MoE Architecture

PYTHON

```python
class GlmMoeDsaMoE(nn.Module):
    def forward(self, hidden_states: torch.Tensor):
        # 1. Router assigns tokens to top-k experts
        router_logits = self.gate(hidden_states)
        indices, weights = self.route_tokens(router_logits) # Sigmo

        # 2. Process through selected routed experts (sparse)
        routed_out = self.experts(
            hidden_states, indices, weights
        )

        # 3. Process through shared expert (dense, every token)
        shared_out = self.shared_experts(hidden_states)

        # 4. Combine outputs
        return routed_out + shared_out
```

18

Summary

19

All Interactive Demos

🚀 **Interactive Demo:** dsa_demo.html

🚀 **Interactive Demo:** moe_demo.html

20

Lecture Summary

- **GLM-5** is a 744B-parameter MoE model with 40B active params, trained on Huawei Ascend chips.
- **DSA** (from DeepSeek): Indexer network selects top-2048 tokens, enabling 200K context windows efficiently.
- **MLA** (from DeepSeek): Compresses KV cache to a fraction of its size using low-rank projections.
- **MoE**: Routes tokens to 8 of 256 independent experts via sigmoid gating, plus 1 shared expert.
- **Unique to GLM-5**: Hardware sovereignty, Slime async RL, Multi-Token Prediction, and agentic engineering focus.

21

Supplementary Resources

🚀 **Interactive Demo:** handout.html

22